

Hebrew Acronym Expansion in Context: Free Generation and Candidate Selection

Ben Carmel Yair Ben David Shaked Schnarch

Tel Aviv University

NLP course project — working draft, 3 October 2026

Abstract

A language model can interpret an acronym in a sentence by generating an expansion or by choosing from an inventory. We study this distinction for Hebrew using five systems: a fine-tuned DictaBERT candidate ranker, and generation and selection configurations of Qwen and Gemini. The comparison changes both candidate access and answer format. The available data contain 2,829 training items and 62 natural development items, with construction methods kept identifiable. Each of the 11 development acronym types has a single stored gold label; this does not establish that its contexts share one meaning. Label defects, candidate coverage and incomplete separation of documents and morphological families limit interpretation. We describe the systems, development scoring and diagnostic findings that inform the evaluation of generation against selection.

1 Introduction

A Hebrew acronym is a short written form that can represent more than one expression. The task in this study is to supply the expansion of a particular occurrence in a sentence. Recognizing the letters is insufficient when several expansions are possible; the system must relate the marked occurrence to the surrounding text. Orthographic variation and attached prefixes also matter because the target seen in a sentence need not be identical to a dictionary entry.

There are two practical ways to ask a language model for an expansion. In *free generation*, the model receives the sentence and target and writes an answer. In *candidate selection*, it also receives a list of expansions and returns the identifier of one entry. Generation can express an answer missing from the list, but it can also produce an unsupported expansion or identify a referent without expanding the acronym. Selection restricts the

response space and makes unfamiliar candidates available. It therefore depends on the quality and coverage of the inventory, as well as the model’s ability to choose appropriately.

Our research question is: *How does performance on the same Hebrew acronym occurrences differ between free expansion and selection from an inventory?* We compare the two formulations within each of two language models, Qwen and Gemini. A Hebrew encoder, DictaBERT, provides a complementary supervised selection system. It scores sentence–candidate pairs after task-specific training. The available train and dev files separate exact stored acronym forms, but not all morphological families. All five arms receive the sentence and the same identified target occurrence. There is no condition without the sentence.

This design addresses a choice of task formulation, not a causal explanation of model behavior. The LLM comparison changes the information supplied and the required response, from a phrase to a letter. The encoder comparison additionally changes pretraining, architecture, supervision and decoding. A performance difference cannot be attributed solely to model size, candidate information or use of context. Nor does separation of stored acronym forms establish separation of their morphological families or absence from pretraining.

2 Related Work

Hebrew acronym interpretation has an established research history. HAADS addresses abbreviation disambiguation in Hebrew and Aramaic Jewish Law documents (HaCohen-Kerner et al., 2010). For Modern Hebrew, Jacobs et al. (2020) develop an acronym dictionary from unannotated text and enrich it with contextual information for disambiguation. Their approach also addresses cases in which an expansion is absent from the acronym’s

document. These studies establish that both collecting possible expansions and choosing among them are substantive problems. Our study starts with an identified target occurrence and an existing candidate inventory. It therefore evaluates expansion and selection under that input contract, rather than end-to-end acronym detection or dictionary construction. The quality and coverage of the supplied inventory remain part of the conditions under which a selection score can be interpreted.

A related distinction concerns what an evaluation split measures. [Chen et al. \(2023\)](#) introduce GLADIS, an English acronym benchmark covering general, scientific, and biomedical domains. Its task splits keep training acronym types out of validation and test. Their AcroBERT model jointly encodes a context and a candidate expansion, using a triplet objective to distinguish matching from nonmatching pairs. This provides a precedent for scoring candidate text and transferring across task-training types. Our DictaBERT ranker uses pairwise binary classification, so it is not a reproduction of AcroBERT’s training procedure.

Data construction also affects this comparison. GLADIS builds its general and biomedical evaluation sets by replacing annotated entity long forms with acronyms, while repartitioning an existing acronym dataset for the scientific domain ([Chen et al., 2023](#)). Replacement creates a labeled example without requiring a naturally occurring acronym mention. However, the sentence was originally written with the full expression. For our study, this distinction motivates retaining the construction history of each example and distinguishing largely substituted training material from natural development occurrences. It does not establish that one source is necessarily easier, since the acronym and expansion distributions also differ.

Generation introduces a separate evaluation problem: the answer may express the right expansion without matching the reference string. [Agrawal et al. \(2022\)](#) prompt an LLM to expand abbreviations in clinical snippets without showing answer choices. They then map its output to the candidate with the greatest contiguous character overlap. Their design separates the information available to the generator from the inventory available to the evaluator. It also shows why a reported generation score depends on the answer resolver. We do not assume that this English clinical matching rule is appropriate for Hebrew, where acceptable wording and the distinction between expand-

ing a name and identifying its referent require explicit treatment.

The same concern appears in word sense disambiguation. [Meconi et al. \(2025\)](#) study selection among supplied definitions and separately ask LLMs to generate definitions, explanations, and examples without a predefined answer list. Their generation study uses expert judgments rather than treating literal agreement with a dictionary definition as sufficient. Word definitions and acronym expansions are different outputs, but the methodological lesson is relevant: the two response formats require a stated basis for comparison. Our selection metrics are implemented, while a final human judgment protocol for free expansions remains to be completed.

3 Task and Data

3.1 The unit of prediction

An item is a tuple $(a_i, x_i, s_i, r_i, C_i, y_i)$: an acronym type, a sentence, a half-open character span, its exact raw text, a candidate inventory, and a reference expansion. The span identifies one occurrence even when the sentence contains several abbreviations. We preserve $r_i = x_i[s_i]$, including an attached prefix, and mark that span with [ACR] and [/ACR]. The type is a grouping key; it is not substituted for the raw target in the input.

Generation predicts a short expansion from (x_i, s_i, r_i) , without candidates or the reference. Selection predicts one member of C_i from the same marked sentence and the supplied inventory. The reference label is used for supervised training and evaluation. Selection includes the reference among its candidates but does not identify which answer is correct. The intended answer is a linguistic expansion appropriate to the occurrence. An entity description or a plausible etymology alone is insufficient. Some named uses require an explicit annotation decision: for example, the current development labels accept רמב"ם for רבי משה בן מימון, including institutional uses. This is a recorded local convention, not a general rule for all names.

Table 1 illustrates the interface. It is a constructed explanation, not a dataset item or model output.

3.2 Sources and supervision

The retained collection draws on Hebrew Wikipedia and Knesset proceedings. Candidate evidence was collected from Wikipedia abbrevi-

Sentence	ח"כ נשא נאום במליאה.
Target	ח"כ (the first token)
Generation	A short expansion, e.g. חבר כנסת
Selection	A letter assigned to one supplied candidate

Table 1: Illustrative input and answer formats. The sentence means “A member of parliament gave a speech in the plenum.” The target is shown separately here for readability; both tasks receive it marked inside the sentence. The example makes no claim about system accuracy.

Quantity	Train	Dev
Occurrences	2,829	62
Acronym types	434	11
Nonempty document keys	1,864	35
Sentence–candidate pairs	12,684	331
Substituted Wikipedia	2,316	0
Natural Wikipedia	50	0
Natural Knesset	319	62
Deglossed Wikipedia	8	0
Claude-authored	136	0

Table 2: Available training and development data. Source and construction categories follow stored metadata. Every item has at least two distinct candidates.

ation and disambiguation pages and Wiktionary. Sentence and label sources are distinct: neither natural occurrence nor dictionary membership establishes the correct expansion in context.

Most training examples are *substituted*: a spelled-out expansion in a Wikipedia sentence was replaced with the acronym, and the removed expression supplies a weak label. Natural examples retain an observed acronym; a small *deglossed* subset removes an adjacent explanation. The current release also contains Claude-authored examples. Substitution and authorship supply supervision cheaply but need not reproduce naturally abbreviated text. Stored labels were reused without comprehensive reannotation.

Table 2 summarizes the available training and development files. The 136 Claude-authored items have no source document and are excluded from the document-key count.

All development items are natural Knesset occurrences. Of their labels, 42 reuse content-matched historical review and 20 reflect AI-assisted author decisions; this is not independent double annotation. Training metadata identifies 2,507 labels without independent verification during preparation, 316 with historical review evidence and six with AI-assisted author decisions. File identities and detailed annotation provenance

are provided in the paper’s technical documentation.

3.3 Qualification and separation

Preparation checks item identity, target boundaries and candidate structure. Targets use a unique bounded match after limited quotation-mark folding or a recorded human span. Each pair-loss item requires at least two distinct candidates and exactly one candidate matching the reference after surrounding whitespace is trimmed. These checks establish structural validity, not semantic correctness.

The available train and dev files have no identical sentence strings or exact acronym forms in common. However, three document keys are shared, involving three training and five development items. The inspected data therefore lack complete document separation. This observation does not measure an effect on model performance or establish overlap with test.

Separation of stored forms also leaves morphological overlap. Training contains מדר in אב, while dev contains בד; both include a prefix attached to the doctor abbreviation. Excluding the unprefixed דר from training does not remove all members of that family. Accordingly, the split does not establish generalization to wholly unseen acronym families.

Each development acronym type has a single stored gold label. This annotation pattern does not establish that all its contexts express the same meaning. The current labels and their known defects limit reliable assessment of within-type sense discrimination.

4 Systems and Experimental Design

Table 3 defines the five arms. For a given LLM, generation and selection use the same configured model. Across LLMs, the prompt text is shared within each task, and selection uses the same displayed candidate order for each item. The implemented order is a seeded shuffle of the stored lists; it is recorded with each response. These controls align inputs but do not make architectures, training or inference equivalent.

4.1 Supervised Hebrew candidate ranker

DictaBERT is a pretrained Hebrew encoder (Shmidman et al., 2023). Our implementation is a *cross-encoder*: the marked sentence and one candidate are encoded jointly, allowing attention be-

System	Candidates	Output
DictaBERT ranker	Yes	Top candidate
Qwen generation	No	Expansion text
Qwen selection	Yes	Candidate letter
Gemini generation	No	Expansion text
Gemini selection	Yes	Candidate letter

Table 3: All arms receive a sentence and marked target. Only DictaBERT receives task-specific supervised training.

tween their tokens. A dropout layer and a linear head turn the pooled representation into a scalar score. The model uses the initial classification token (CLS). Candidates are scored independently, and prediction chooses the largest score within the item’s inventory.

For each training occurrence, the reference candidate is positive and the other supplied candidates are negative. The implemented objective is binary cross-entropy on the candidate-pair logits:

$$\mathcal{L} = -\frac{1}{M} \sum_{j=1}^M [z_j \log \sigma(u_j) + (1 - z_j) \log(1 - \sigma(u_j))], \quad (1)$$

where M is the number of pairs, u_j is a score, σ is the sigmoid function and $z_j \in \{0, 1\}$ is the label. Thus occurrences with larger inventories contribute more pairs. This is binary pair training, not a multiclass softmax over each inventory. AdamW updates the encoder and scoring head; no parameter-efficient adapter is used in this implementation.

The checkpoint is attributed by the authors to the Colab training procedure based on dicta-il/dictabert. That notebook specifies dropout 0.1, learning rate 2×10^{-5} , pair batch size 16, one epoch and seed 42. It sets the seed after model initialization, so initialization is not controlled by that setting. Checkpoints are selected by strict improvement in development pair loss, which differs from occurrence-level selection accuracy.

The pair layout places the marked context before the candidate, separated by BERT special tokens, with a nominal limit of 256 tokens. Context is cropped from the outside while retaining the target and candidate. Candidate length can affect the context budget; the LLMs receive the full sentence. The supplied checkpoint was saved in fp16 and loaded in full, including the encoder and scoring

head, for fp32 development inference. Loading checks support reconstruction but do not authenticate the original training inputs or executed settings. The actual seed and complete training record remain unresolved; notebook settings above are not a verified execution log. Detailed loading evidence and remaining provenance gaps are documented in the repository.

4.2 Prompted language models

The generation prompt, written in Hebrew, asks for the expansion of the marked acronym in a few words, without explanation. The selection prompt supplies lettered candidates and requests exactly one letter. Neither contains demonstrations or identifies the correct answer. Full prompt templates belong to the technical companion; their information content and response requirements are the experimental distinction described here.

The development runs use qwen2.5:7b from the Qwen2.5 family (Qwen Team, 2025), through Ollama with Q4_K_M quantization, and gemini-3.8-flash with thinking level low. Qwen’s server digest is consistent across both tasks; successful Gemini responses preserve the returned version string. No additional Qwen decoding options were supplied, and omitted service defaults remain unresolved. These settings do not establish deterministic decoding; low thinking does not mean disabled thinking.

Requests have a 120-second timeout. The basic path has no automatic retry; Gemini continuation permits up to three attempts per item for transient errors, with backoff and saved progress. Completed responses are retained. The saved runs contain all 62 DictaBERT selections and both sets of 62 Qwen responses; Gemini coverage is partial after HTTP 429 and 503 errors. Prompts, candidate orders, raw answers, model identities and failure statuses are preserved in the [development records](#).

5 Evaluation

5.1 Roles of the data splits

Training supplies supervised candidate pairs. Development supports checkpoint selection and diagnostics of inputs, services and scoring. It is therefore distinct from an independent test evaluation. Split claims refer to the stored forms and documents inspected in Section 3.3.

5.2 Development selection measures

The common evaluator scores all three selection systems on the same requested item IDs. A completed LLM answer must be one uppercase letter in the displayed range after surrounding whitespace is removed; “A.” fails this strict parser. The letter maps through the saved candidate order. A selection is correct when its candidate exactly matches the stored gold label after trimming, without semantic aliases or quotation normalization.

Let c_i indicate a correct selection, N be the number of requested occurrences, A the set of observed types and I_a the occurrences of type a . The implemented summaries are

$$\text{Micro} = \frac{1}{N} \sum_{i=1}^N c_i, \quad (2)$$

$$\text{Macro} = \frac{1}{|A|} \sum_{a \in A} \frac{1}{|I_a|} \sum_{i \in I_a} c_i. \quad (3)$$

Micro weights occurrences equally; macro weights stored acronym types equally. Missing labels make accuracy unavailable, and missing type metadata makes macro unavailable. Unavailable values are not zero.

For an attempted arm, errors, incomplete responses, parsing failures and unattempted occurrences remain in the requested denominator with $c_i = 0$. An entirely unrun arm is unmeasured. Counts distinguish service failures (no completed usable response), format failures (completed but invalid output), valid label mismatches and unattempted items. On a partial run, the all-item score combines service coverage and selection success. It is not a clean estimate of linguistic correctness and does not justify ranking partial Gemini execution against complete runs. Restricting scores to successful requests would answer a different question and does not replace the implemented denominator. Label defects further separate reference matching from semantic correctness.

5.3 Generation and paired comparison

Free responses are saved but remain semantically unjudged. A human judgment protocol must define acceptable paraphrases, attached prefixes, named uses, ambiguity and disagreement. Exact string matching can reject equivalent expansions, while inventory membership alone neither ensures contextual correctness nor covers missing valid answers.

The generation–selection comparison pairs the same occurrences within each LLM under a common correctness criterion. A paired table can distinguish both-correct, selection-only, generation-only and neither-correct outcomes. Missing responses must remain visible, rather than restricting the comparison to successful requests. Such a comparison requires agreed judgments and cannot be inferred from completion counts.

The evaluation requires relevant baselines and an account of randomness in training, prompting and decoding. The baseline and method for assessing variability have not yet been selected. This does not imply a requirement for a particular confidence interval, significance test or number of seeds, and none is reported here. Repeated occurrences share types and documents, which also limits independence assumptions. The [study notebook](#) and [shared evaluator](#) provide the implementation details.

6 Results and Discussion

Draft note: final test results and discussion have not yet been incorporated.

7 Limitations

Training is dominated by substituted Wikipedia examples and includes AI-authored text, whereas development is natural parliamentary text. Domain, construction and acronym composition differ together. Document overlap and shared morphological families (Section 3.3) limit separation claims; pretraining exposure is also unknown.

Development has only 62 items across 11 unevenly represented types. Five singleton types contribute five elevenths of macro accuracy, while the largest type contributes 15 of 62 items to micro accuracy. A single stored gold label per type does not demonstrate semantic uniformity: known annotation defects prevent reliable inference about the true sense distribution. The sample and labels therefore cannot establish stable performance across types or robust contextual sense discrimination.

Inspection exposed concrete development-label and inventory defects. One מ"מ sentence explicitly denotes המרכז לטכנולוגיה חינוכית (the educational technology center) but is labeled מטבע חוץ (foreign currency). All nine המ"מ inventories omit חומרים מסוכנים (hazardous materials), including contexts that spell it out; matching the stored

label can therefore reward an unsuitable expansion. These observations can both depress and inflate label-based accuracy. They are findings about the inspected dev items, not evidence of the same defects in test.

Named uses require a separate distinction. The recorded decision accepts רבי משה בן מימון for רמב"ם even in hospital and street contexts. A hospital description may identify a referent without meeting that expansion convention. Disagreement on these items is therefore not automatically a labeling error, and a rule should not be changed retrospectively to favor a system.

Candidate access is coupled to response format, and the encoder and LLMs differ in supervision and architecture. Strict parsing can reject a clear intended selection, while an inventory may omit a valid expansion. These comparisons do not isolate the effect of candidate information or model size. A per-form training-majority baseline is undefined for exact forms absent from training; a relevant alternative still needs to be specified.

AI Disclosure and Reflection

OpenAI Codex (a GPT-6-based assistant) substantially assisted implementation, software checks, literature checking, manuscript drafting and editing, and PDF inspection. AI assistance also informed recorded author annotation decisions. The 136 Claude-authored training examples are identified separately from natural text. Software and presentation checks are distinct from the separately recorded DictaBERT, Qwen and Gemini development runs. Detailed assistance and annotation provenance are documented in the repository. Author contributions and personal reflections remain to be completed by the authors.

References

Monica Agrawal, Stefan Hegselmann, Hunter Lang, Yoon Kim, and David Sontag. 2022. [Large language models are few-shot clinical information extractors](#). In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 1998–2022, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Lihu Chen, Gaël Varoquaux, and Fabian M. Suchanek. 2023. [GLADIS: A general and large acronym disambiguation benchmark](#). In *Proceedings of the 17th Conference of*

the European Chapter of the Association for Computational Linguistics, pages 2073–2088, Dubrovnik, Croatia. Association for Computational Linguistics.

Yaakov HaCohen-Kerner, Ariel Kass, and Ariel Peretz. 2010. [HAADS: A Hebrew Aramaic abbreviation disambiguation system](#). *Journal of the American Society for Information Science and Technology*, 61(9):1923–1932.

Kayla Jacobs, Alon Itai, and Shuly Wintner. 2020. [Acronyms: identification, expansion and disambiguation](#). *Annals of Mathematics and Artificial Intelligence*, 88:517–532. First published online 6 December 2018.

Domenico Meconi, Simone Stirpe, Federico Martelli, Leonardo Lavallo, and Roberto Navigli. 2025. [Do large language models understand word senses?](#) In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33897–33916, Suzhou, China. Association for Computational Linguistics.

Qwen Team. 2025. [Qwen2.5 technical report](#). *Preprint*, arXiv:2412.15115. Version 2, 3 January 2025; first posted December 2024.

Shaltiel Shmidman, Avi Shmidman, and Moshe Koppel. 2023. [DictaBERT: A state-of-the-art BERT suite for modern Hebrew](#). *Preprint*, arXiv:2308.16687. Version 2, 13 October 2023.